

Concordia University

Lab 4: Introduction to Simulink and Filter Design

ELEC 342
Lab Section: UL-X

Discrete Time Signals and Systems

Penoelo Thibaud
40212017

Lab Instructor: Razieh Abdolahi

Performed on : November 5th, 2024
Submitted on : November 19, 2024

I certify that this submission is my original work and meets the Faculty's Expectations of Originality

Objectives –

This lab aims to introduce concepts such as upsampling, downsampling, and the creation and manipulation of audio signals, while emphasizing the practical implementation of digital and analog filters. Additionally, the lab familiarizes students with tools like FDATool and SPTool to design, analyze, and visualize filters and signals effectively. Through hands-on exercises, the lab bridges theoretical concepts with practical applications in dynamic simulation environments.

Theory -

Signal Processing Fundamentals:

Signal processing involves the analysis and manipulation of discrete and continuous signals.

Sampling, upsampling and downsampling:

Affect the resolution and frequency content of signals.

Nyquist-Shannon sampling theorem:

the sampling rate must be at least twice the highest frequency in the signal to avoid aliasing.

Filter Types and Characteristics

Filters are used to selectively attenuate or allow specific frequency ranges within a signal. Common filter types include:

Low-Pass Filters:

Pass frequencies below the cutoff while attenuating higher frequencies.

High-Pass Filters:

Pass higher frequencies and attenuate lower ones.

Band-Pass Filters:

Allow frequencies within a specific range while attenuating others.

Band-Stop and Notch Filters:

Reject specific frequency ranges or narrow bands.

Filters characteristic:

Parameters such as passband, stopband, cutoff frequency, ripple, and attenuation.

Frequency Domain Analysis:

The Fourier Transform (FFT) is a mathematical tool used to convert signals from the time domain to the frequency domain.

MATLAB Sound Commands:

Functions like sound for playing audio signals.

MATLAB Fourier Transform:

fft for frequency domain analysis.

MATLAB Signal Plotting:

Functions like plot and stem for visualizing signals.

Simulink “From Multimedia File”:

Imports audio data.

Simulink “To Audio Device”:

Plays processed audio signals

Simulink “Upsample” and “Downsample”:

Increase or decrease the sampling rate of signals.

Simulink “Scope”:

Visualizes signals in time and frequency domains.

FIR Filters:

Always stable with linear phase but require higher order for sharp transitions.

IR Filters:

More computationally efficient with nonlinear phase, suitable for applications with strict memory constraints.

Prelab

Questions 1

Answer (1A):

I read the audio from the lab_5_Audio_1.wav file as required.

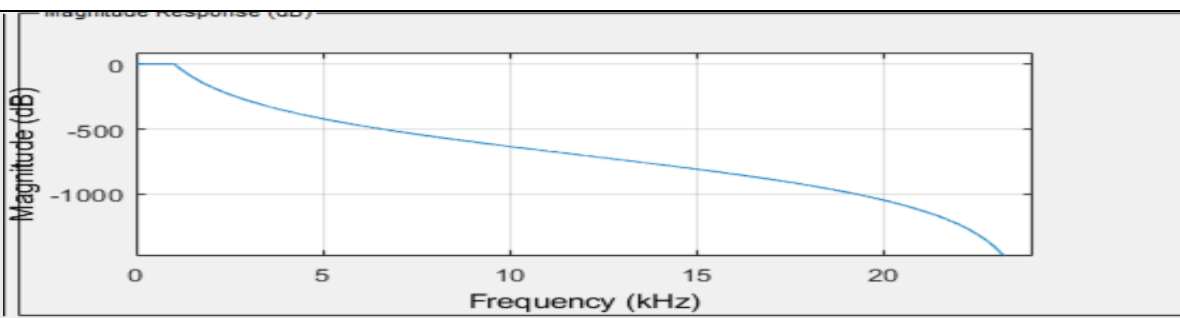
Answer (1B):

The second reading had a lower volume than the first one, since I multiplied the magnitude by 0.25. The third reading had a higher volume than the first and second ones, since I multiplied the magnitude by 4.

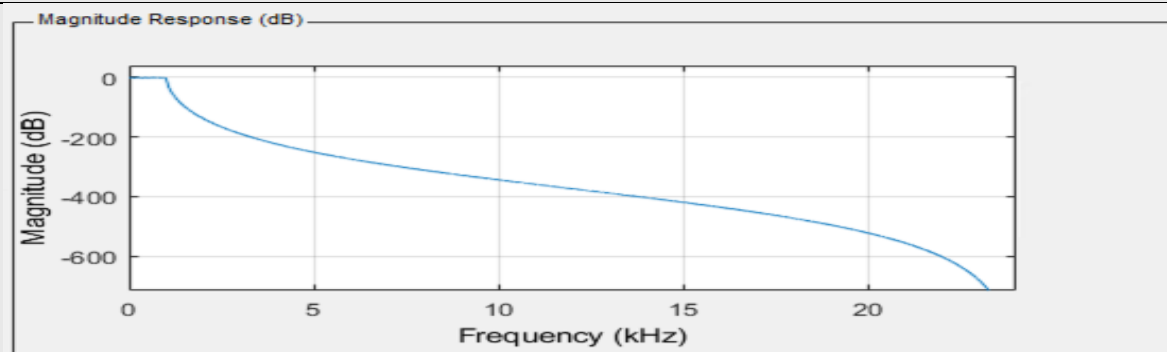
Answer (1C):

The fourth reading played slower than previous reading, since I divided the frequency by 2. The fifth reading played faster than previous reading, since I multiplied the frequency by 2.

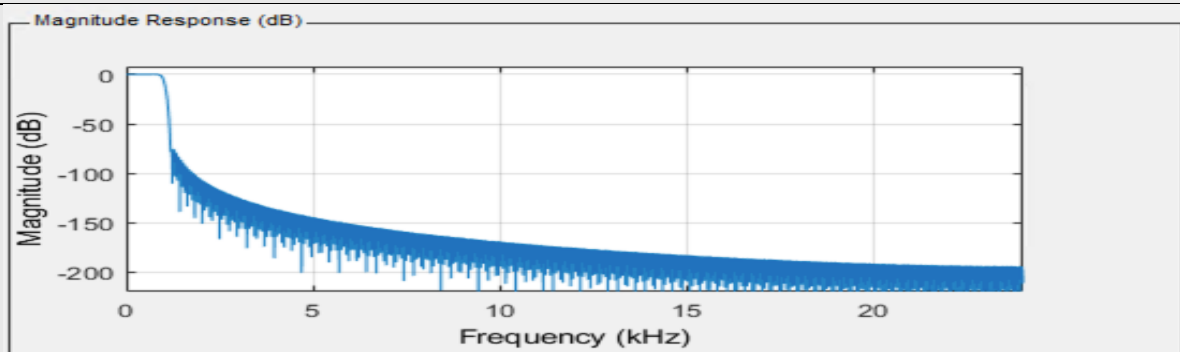
Questions 2



Butterworth filter



Chebyshev Type I filter



FIR filter using the Blackman Window

Answer (2A):

Butterworth filter: 30

Answer (2B):

Chebyshev Type I filter: 13

Answer (2C):

The windowing technique is really inefficient, since it uses 721 coefficients compared to 30 for the Butterworth and 13 for the Chebyshev type I filter. Which make it way 24 to 56 time more memory intensive then the other alternatives and requires way more computation.

Answer (2D):

```
clear

Fs = 8000; % Sampling frequency (8 kHz)
Fpass = 1000; % Passband frequency (1 kHz)
Fstop = 1400; % Stopband frequency (1.4 kHz)
Apass = 1; % Passband ripple (1 dB)
Astop = 80; % Stopband attenuation (80 dB)

% Define filter specifications
h = fdesign.lowpass('fp,fst,ap,ast', Fpass, Fstop, Apass, Astop, Fs);

% Design the Butterworth filter
Hd_butter = design(h, 'butter', 'MatchExactly', 'stopband');

% Calculate the Butterworth filter order
butter_order = order(Hd_butter);
disp(['Butterworth Filter Order: ', num2str(butter_order)]);

% Design the Chebyshev Type I filter
Hd_cheby = design(h, 'cheby1', 'MatchExactly', 'passband');

% Calculate the Chebyshev filter order
cheby_order = order(Hd_cheby);
disp(['Chebyshev Type I Filter Order: ', num2str(cheby_order)]);

% Transition bandwidth
BW = Fstop - Fpass;

% FIR filter order approximation
N = round(6 * Fs / BW);

% Ensure order is odd
if mod(N, 2) == 0
    N = N + 1;
end
```

```

% Design the FIR filter
b_fir = fir1(N, Fpass / (Fs / 2), blackman(N + 1));

% Display the FIR filter order
disp(['FIR Filter Order: ', num2str(N)]);

% Display memory requirements
butter_coeff = butter_order + 1; % For Butterworth and Chebyshev, memory = order + 1
cheby_coeff = cheby_order + 1;
fir_coeff = length(b_fir);

disp(['Memory required for Butterworth: ', num2str(butter_coeff)]);
disp(['Memory required for Chebyshev: ', num2str(cheby_coeff)]);
disp(['Memory required for FIR: ', num2str(fir_coeff)]);

% Relative inefficiency of FIR
inefficiency = fir_coeff / min(butter_coeff, cheby_coeff);
disp(['FIR is ', num2str(inefficiency), ' times more memory-intensive than the best IIR filter.']);

% Round coefficients
[b_butter, a_butter] = tf(Hd_butter);
[b_cheby, a_cheby] = tf(Hd_cheby);

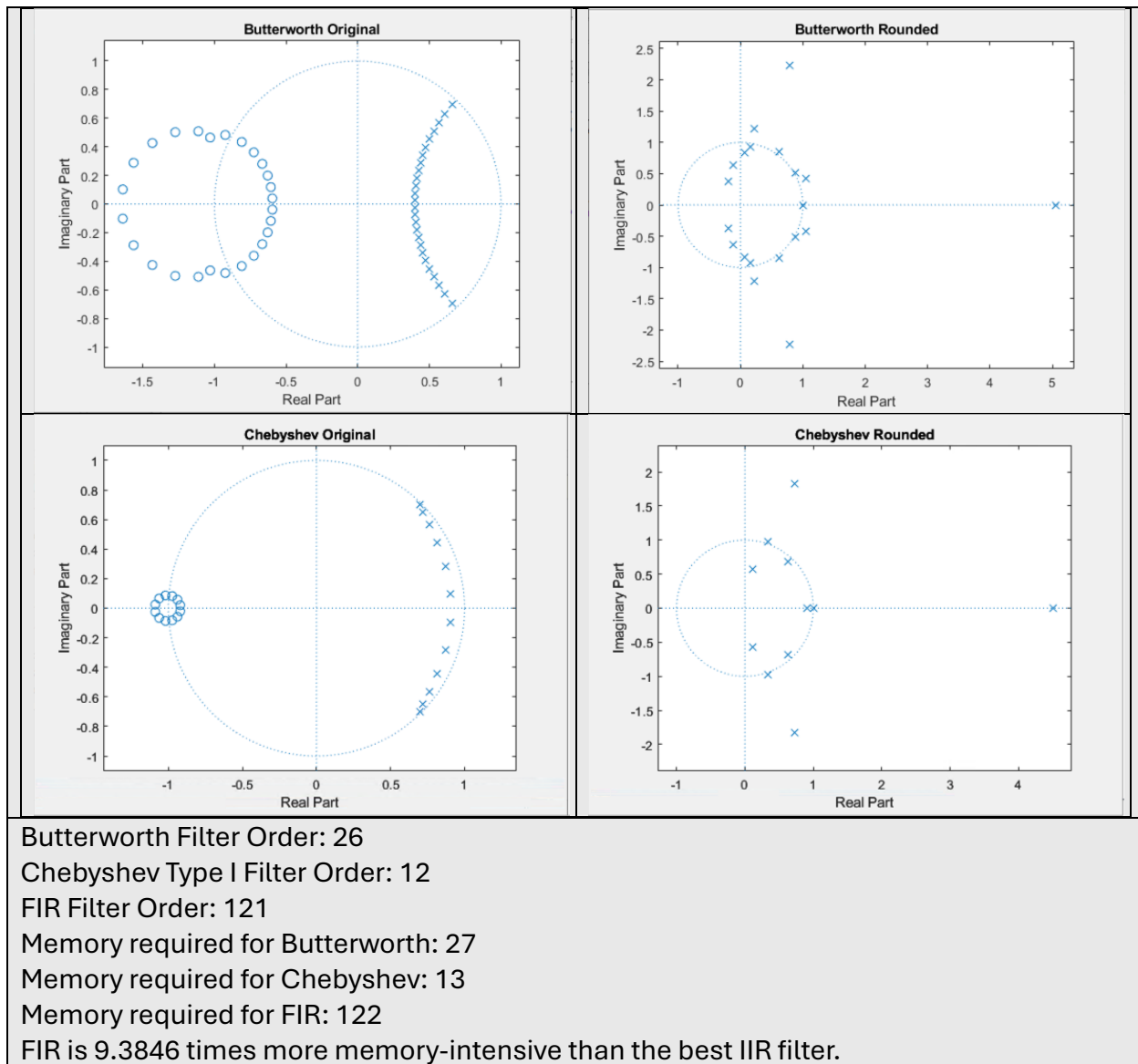
b_butter_rounded = round(b_butter);
a_butter_rounded = round(a_butter);

b_cheby_rounded = round(b_cheby);
a_cheby_rounded = round(a_cheby);

% Pole-Zero plots
figure; zplane(b_butter, a_butter); title('Butterworth Original');
figure; zplane(b_butter_rounded, a_butter_rounded); title('Butterworth Rounded');

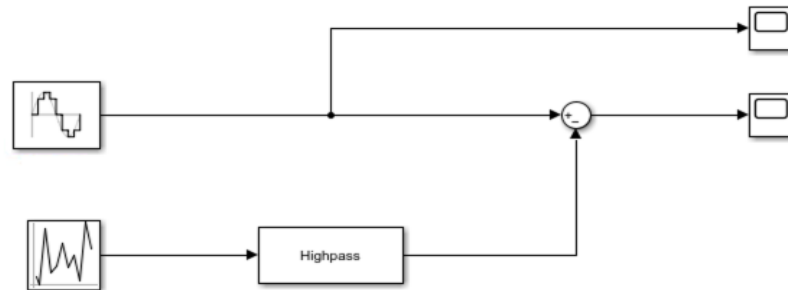
figure; zplane(b_cheby, a_cheby); title('Chebyshev Original');
figure; zplane(b_cheby_rounded, a_cheby_rounded); title('Chebyshev Rounded');

```

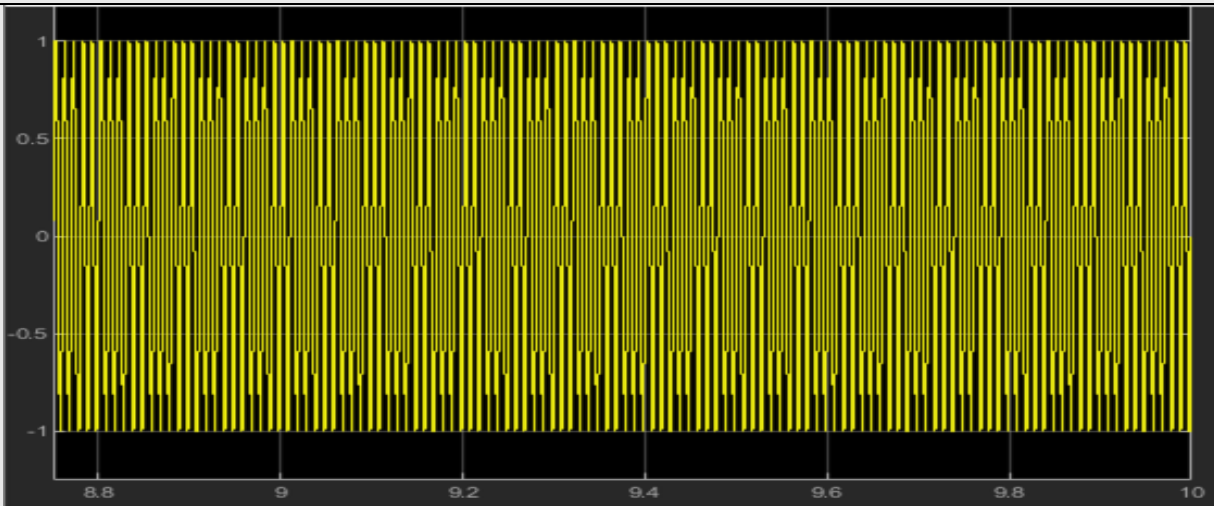



These results highlight the importance of precision in IIR filter coefficients, as small errors can destabilize the system and significantly alter performance.

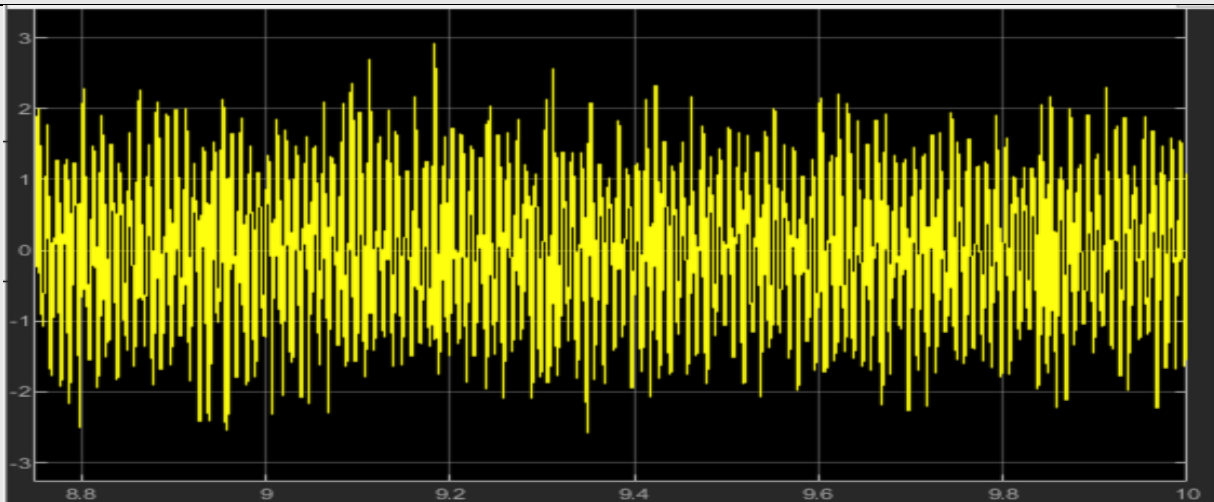
Questions 3



SIMULINK MODEL



CLEAN SIGNAL



NOISY SIGNAL

Tasks/Results/Discussion

Questions 1

Answer (1A):

```
% Sampling frequency
Fs = 16000;
f_pass = 2000; % Passband frequency (Hz)
f_stop1 = 100; % Stopband frequency 1 (Hz)
f_stop2 = 6000; % Stopband frequency 2 (Hz)

% Normalized frequencies
Wn = [f_pass-100, f_pass+100] / (Fs/2); % Narrow band around 2000 Hz

% Design Butterworth filters
[b4, a4] = butter(4, Wn, 'bandpass'); % 4th-order
[b8, a8] = butter(8, Wn, 'bandpass'); % 8th-order

% Generate input signal
t = 0:1/Fs:0.01; % Time vector (10 ms)
x = sin(2*pi*100*t) + sin(2*pi*2000*t) + sin(2*pi*6000*t);

% Filter the input signal
y4 = filter(b4, a4, x);
y8 = filter(b8, a8, x);

% Plot the results
figure;
subplot(3,1,1); plot(t, x); title('Input Signal'); xlabel('Time (s)'); ylabel('Amplitude');
subplot(3,1,2); freqz(b4, a4, 1024, Fs); title('4th-Order Butterworth Magnitude Response');
subplot(3,1,3); freqz(b8, a8, 1024, Fs); title('8th-Order Butterworth Magnitude Response');

figure;
subplot(2,1,1); plot(t, y4); title('4th-Order Butterworth Output'); xlabel('Time (s)'); ylabel('Amplitude');
subplot(2,1,2); plot(t, y8); title('8th-Order Butterworth Output'); xlabel('Time (s)'); ylabel('Amplitude');

% FFT of input and output signals (8th-order)
X = fft(x);
Y8 = fft(y8);
```

```
f = (0:length(X)-1) * Fs / length(X);
```

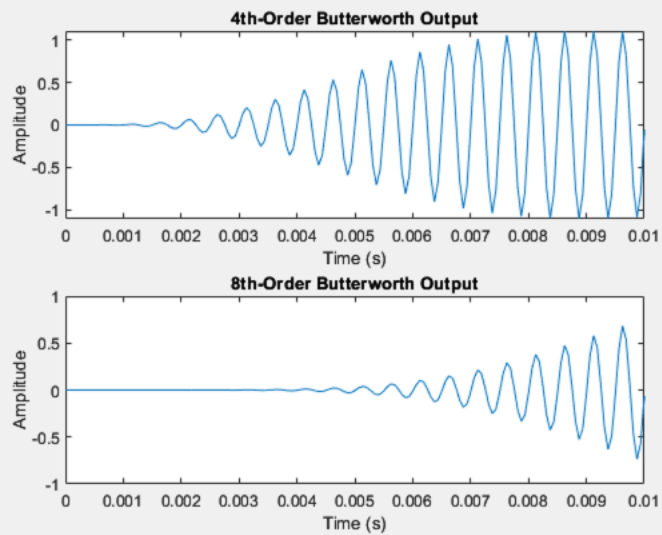
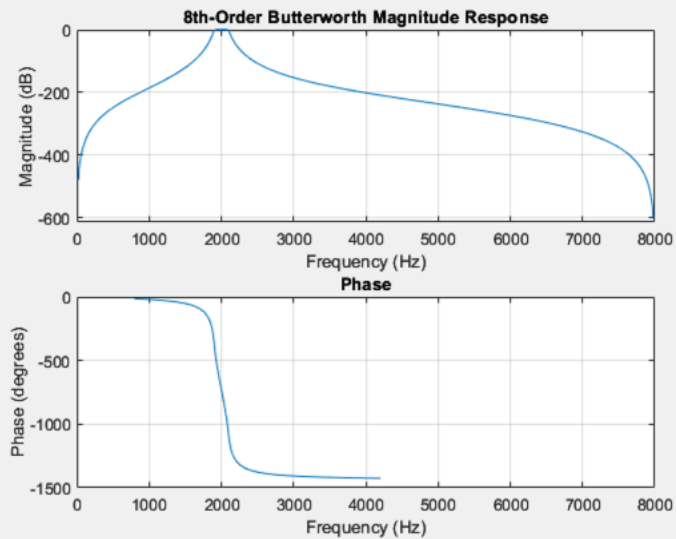
```
figure;
```

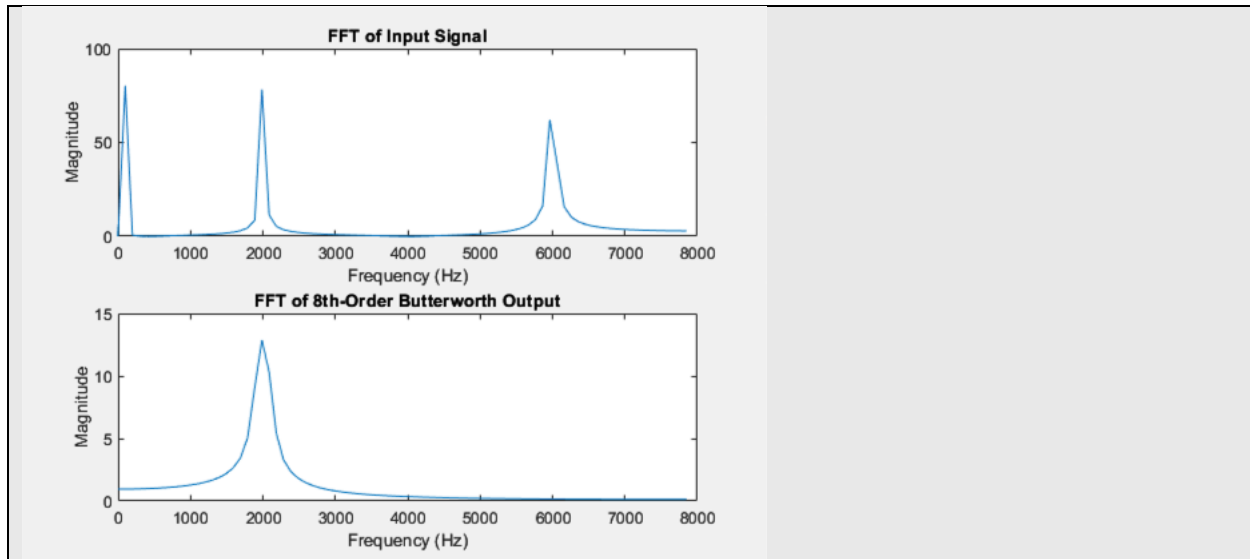
```
subplot(2,1,1); plot(f(1:end/2), abs(X(1:end/2))); title('FFT of Input Signal');
```

```
xlabel('Frequency (Hz)'); ylabel('Magnitude');
```

```
subplot(2,1,2); plot(f(1:end/2), abs(Y8(1:end/2))); title('FFT of 8th-Order Butterworth Output');
```

```
xlabel('Frequency (Hz)'); ylabel('Magnitude');
```





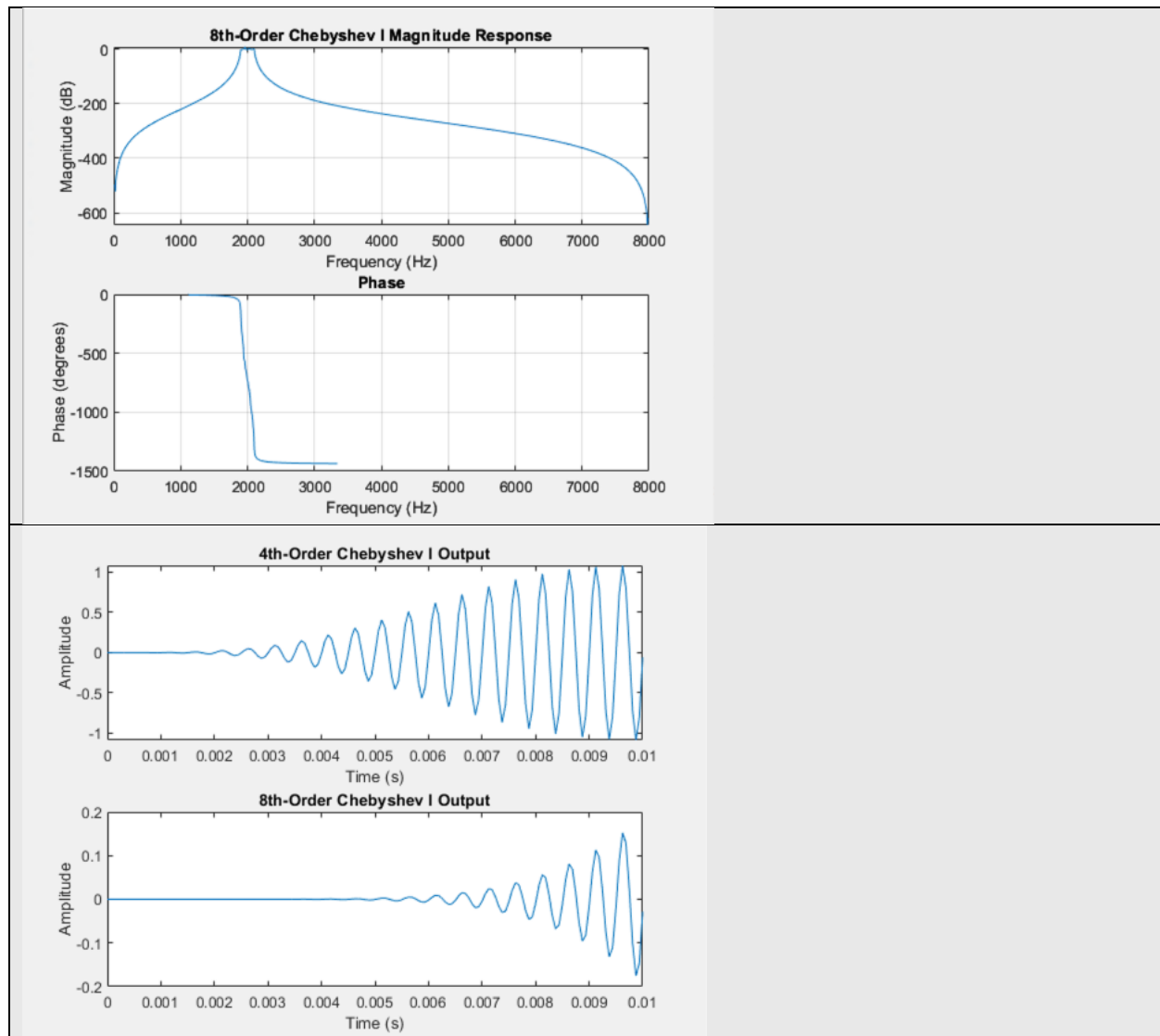
Answer (1B):

```
% Design Chebyshev Type I filters
[b4, a4] = cheby1(4, 1, Wn, 'bandpass'); % 4th-order
[b8, a8] = cheby1(8, 1, Wn, 'bandpass'); % 8th-order

% Filter the input signal
y4 = filter(b4, a4, x);
y8 = filter(b8, a8, x);

% Plot the results
figure;
subplot(3,1,1); plot(t, x); title('Input Signal'); xlabel('Time (s)'); ylabel('Amplitude');
subplot(3,1,2); freqz(b4, a4, 1024, Fs); title('4th-Order Chebyshev I Magnitude Response');
subplot(3,1,3); freqz(b8, a8, 1024, Fs); title('8th-Order Chebyshev I Magnitude Response');

figure;
subplot(2,1,1); plot(t, y4); title('4th-Order Chebyshev I Output'); xlabel('Time (s)'); ylabel('Amplitude');
subplot(2,1,2); plot(t, y8); title('8th-Order Chebyshev I Output'); xlabel('Time (s)'); ylabel('Amplitude');
```



Answer (1C):

```
% Design Elliptic filters
[b4, a4] = ellip(4, 1, 60, Wn, 'bandpass'); % 4th-order
[b8, a8] = ellip(8, 1, 60, Wn, 'bandpass'); % 8th-order

% Filter the input signal
y4 = filter(b4, a4, x);
y8 = filter(b8, a8, x);

% Plot the results
figure;
subplot(3,1,1); plot(t, x); title('Input Signal'); xlabel('Time (s)'); ylabel('Amplitude');
```

```
subplot(3,1,2); freqz(b4, a4, 1024, Fs); title('4th-Order Elliptic Magnitude Response');
subplot(3,1,3); freqz(b8, a8, 1024, Fs); title('8th-Order Elliptic Magnitude Response');
```

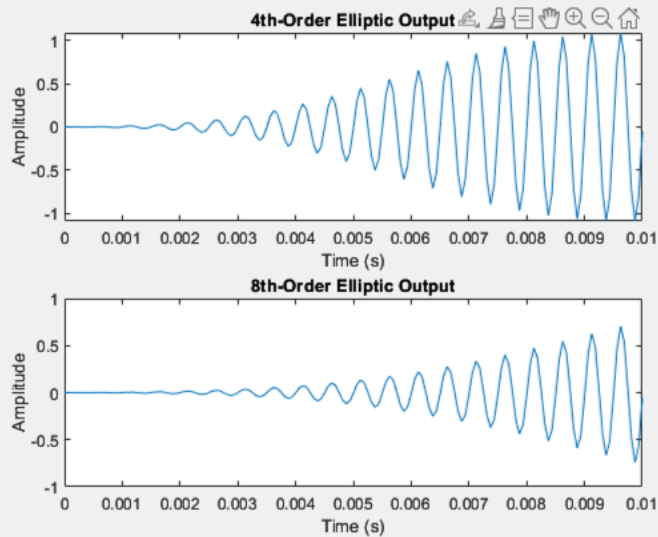
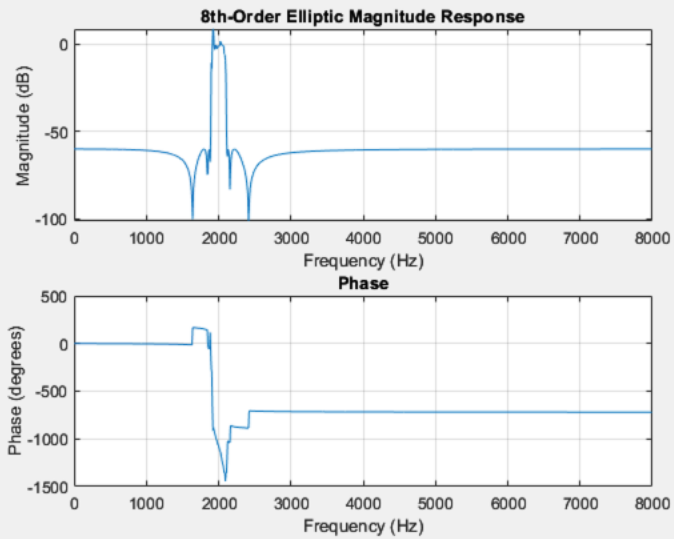
```
figure;
```

```
subplot(2,1,1); plot(t, y4); title('4th-Order Elliptic Output'); xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
subplot(2,1,2); plot(t, y8); title('8th-Order Elliptic Output'); xlabel('Time (s)');
```

```
ylabel('Amplitude');
```



Questions 2

Answer (2A):

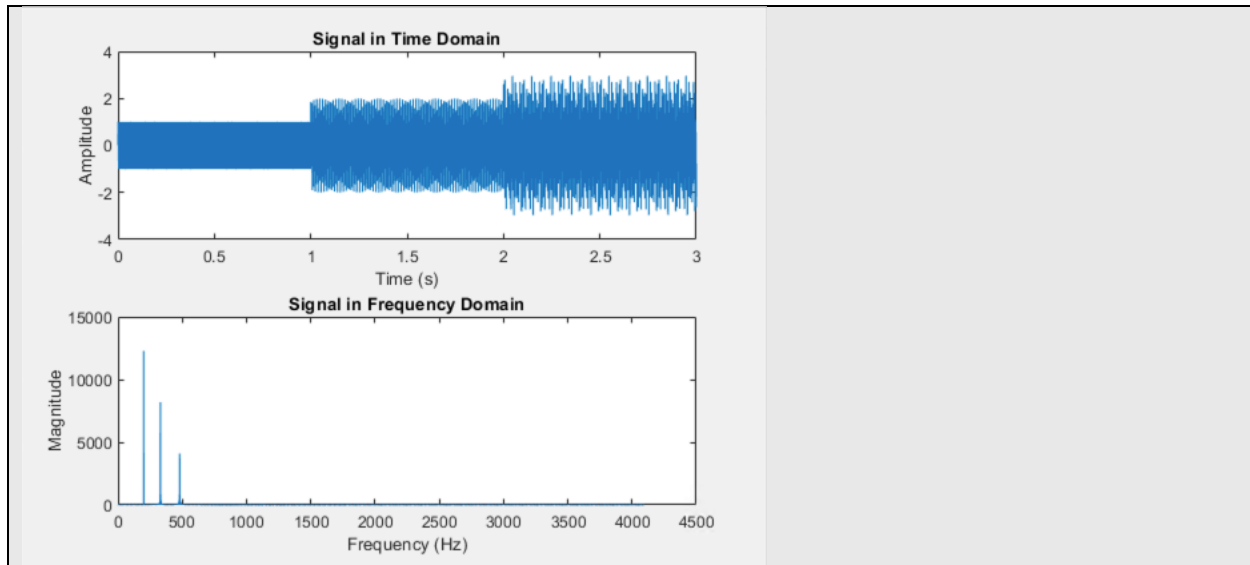
```
fs = 8192;
T = 3;
t = 0:1/fs:T-1/fs;

% Generate components
x = sin(2*pi*200*t);
x(t >= 1) = x(t >= 1) + sin(2*pi*330*t(t >= 1));
x(t >= 2) = x(t >= 2) + sin(2*pi*480*t(t >= 2));

% View signal in time and frequency domain
figure;
subplot(2, 1, 1);
plot(t, x);
title('Signal in Time Domain');
xlabel('Time (s)');
ylabel('Amplitude');

% FFT to view frequency domain
X = fft(x);
f = linspace(0, fs/2, length(X)/2);
subplot(2, 1, 2);
plot(f, abs(X(1:length(f))));
title('Signal in Frequency Domain');
xlabel('Frequency (Hz)');
ylabel('Magnitude');

% Listen to the signal
soundsc(x, fs);
```



Answer (2B):

```
% Design notch filter
wo = 330/(fs/2); % Normalized frequency
bw = wo/35; % Bandwidth
[b, a] = iirnotch(wo, bw);

% Plot filter response
[h, w] = freqz(b, a, fs);
figure;
subplot(2, 1, 1);
plot(w*fs/(2*pi), abs(h));
title('Filter Magnitude Response');
xlabel('Frequency (Hz)');
ylabel('Magnitude');

subplot(2, 1, 2);
plot(w*fs/(2*pi), angle(h));
title('Filter Phase Response');
xlabel('Frequency (Hz)');
ylabel('Phase (radians)');

% Apply filter
x_filtered = filter(b, a, x);

% Compare in time domain
figure;
subplot(2, 1, 1);
```

```

plot(t, x);
title('Original Signal (Time Domain)');
xlabel('Time (s)');
ylabel('Amplitude');

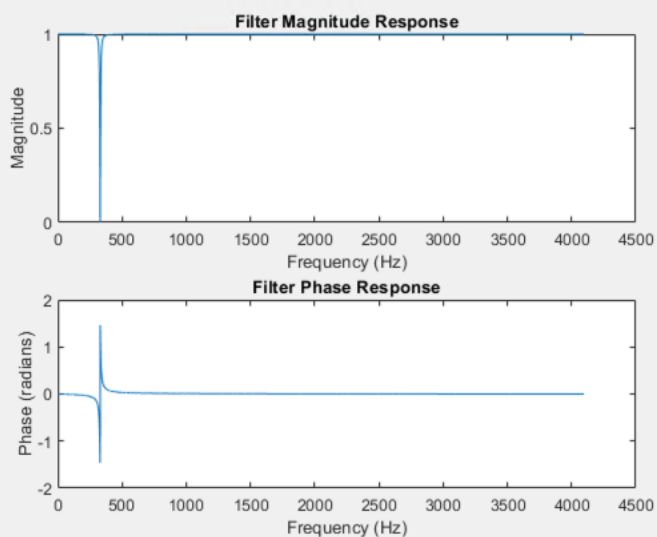
subplot(2, 1, 2);
plot(t, x_filtered);
title('Filtered Signal (Time Domain)');
xlabel('Time (s)');
ylabel('Amplitude');

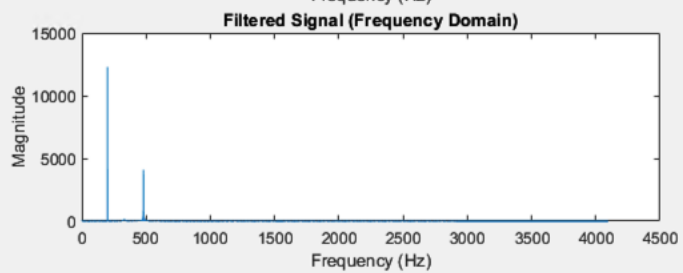
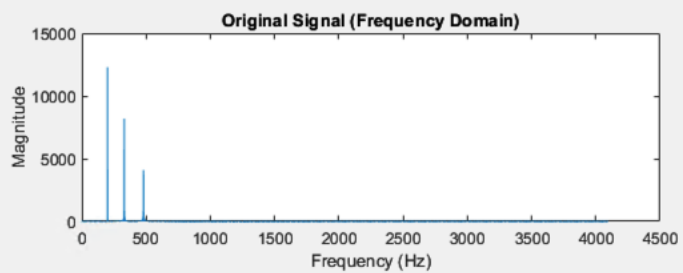
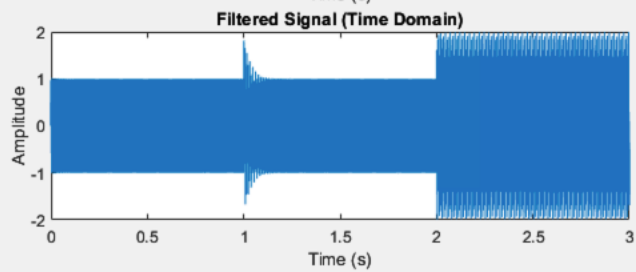
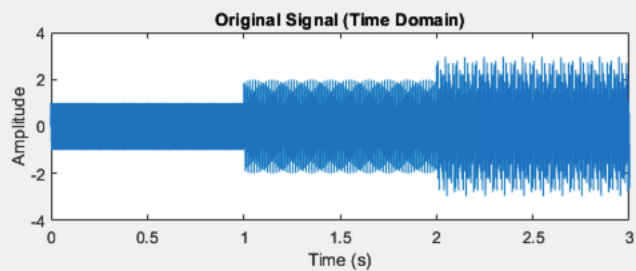
% Compare in frequency domain
X_filtered = fft(x_filtered);
figure;
subplot(2, 1, 1);
plot(f, abs(X(1:length(f))));
title('Original Signal (Frequency Domain)');
xlabel('Frequency (Hz)');
ylabel('Magnitude');

subplot(2, 1, 2);
plot(f, abs(X_filtered(1:length(f))));
title('Filtered Signal (Frequency Domain)');
xlabel('Frequency (Hz)');
ylabel('Magnitude');

% Listen to the filtered signal
soundsc(x_filtered, fs);

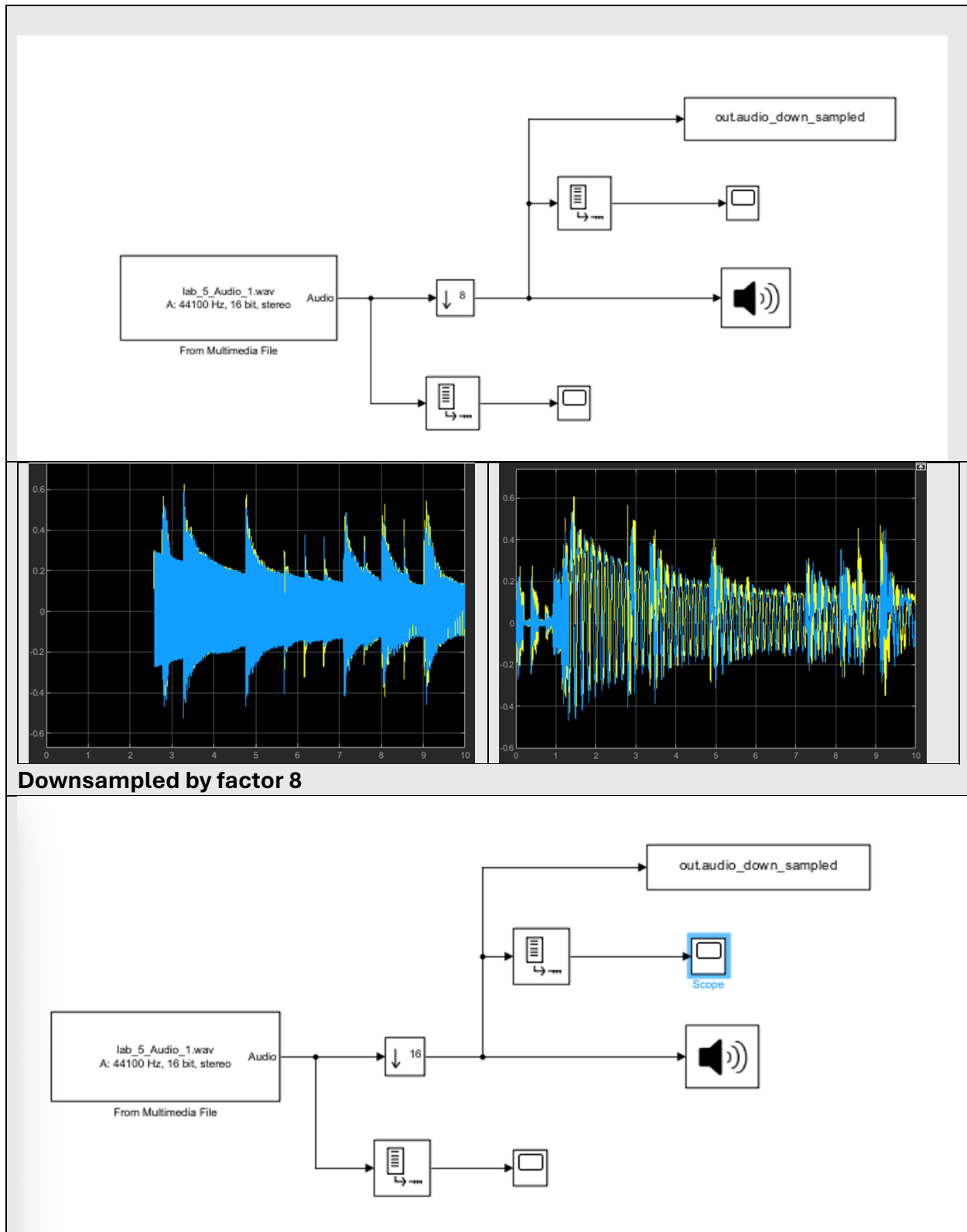
```

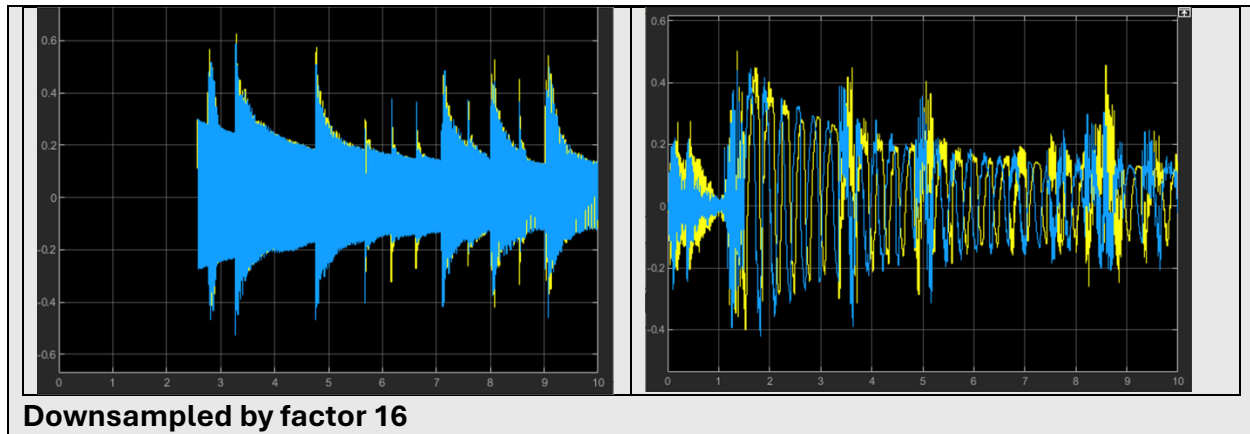




Questions 3

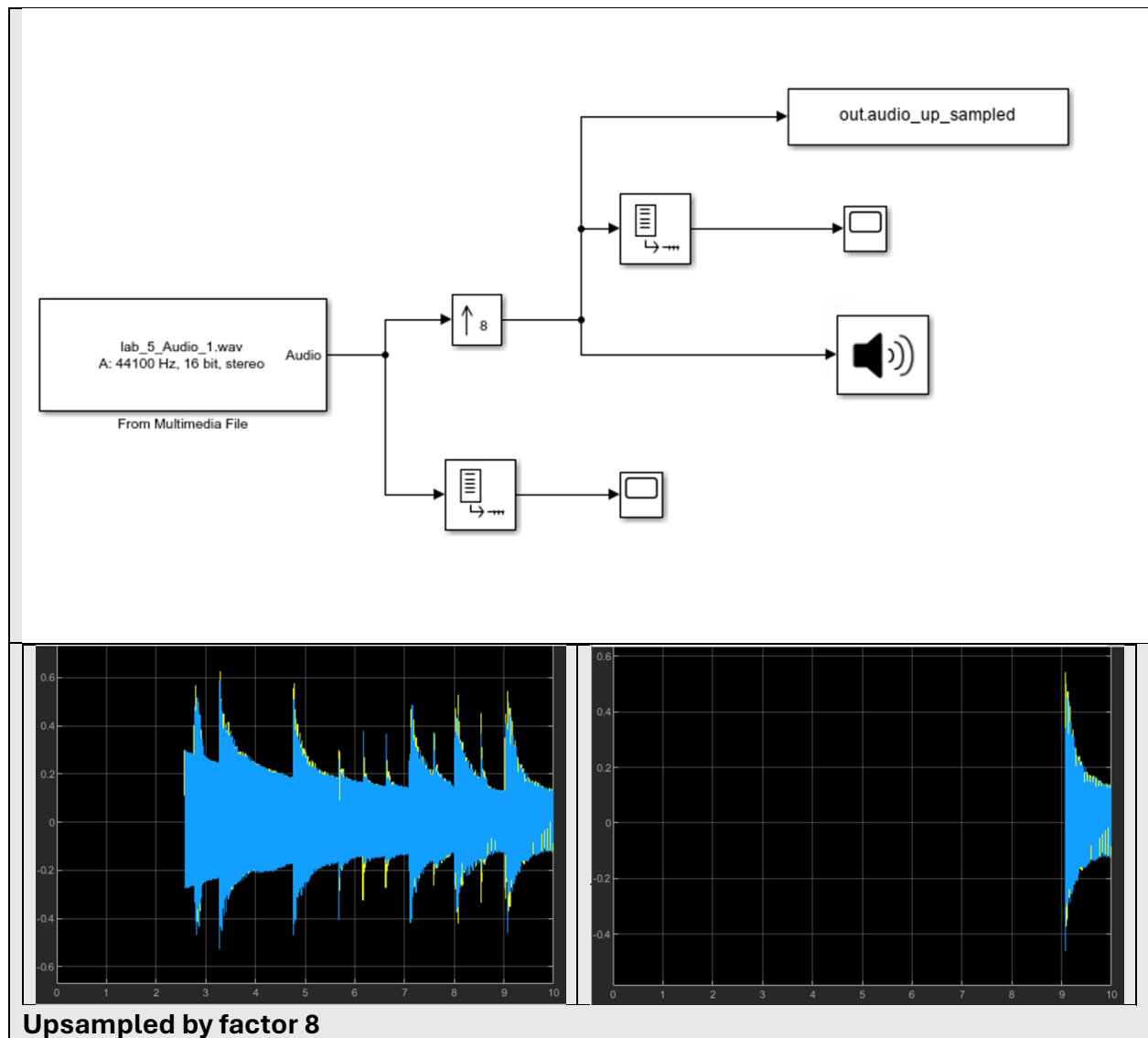
Answer (3A):

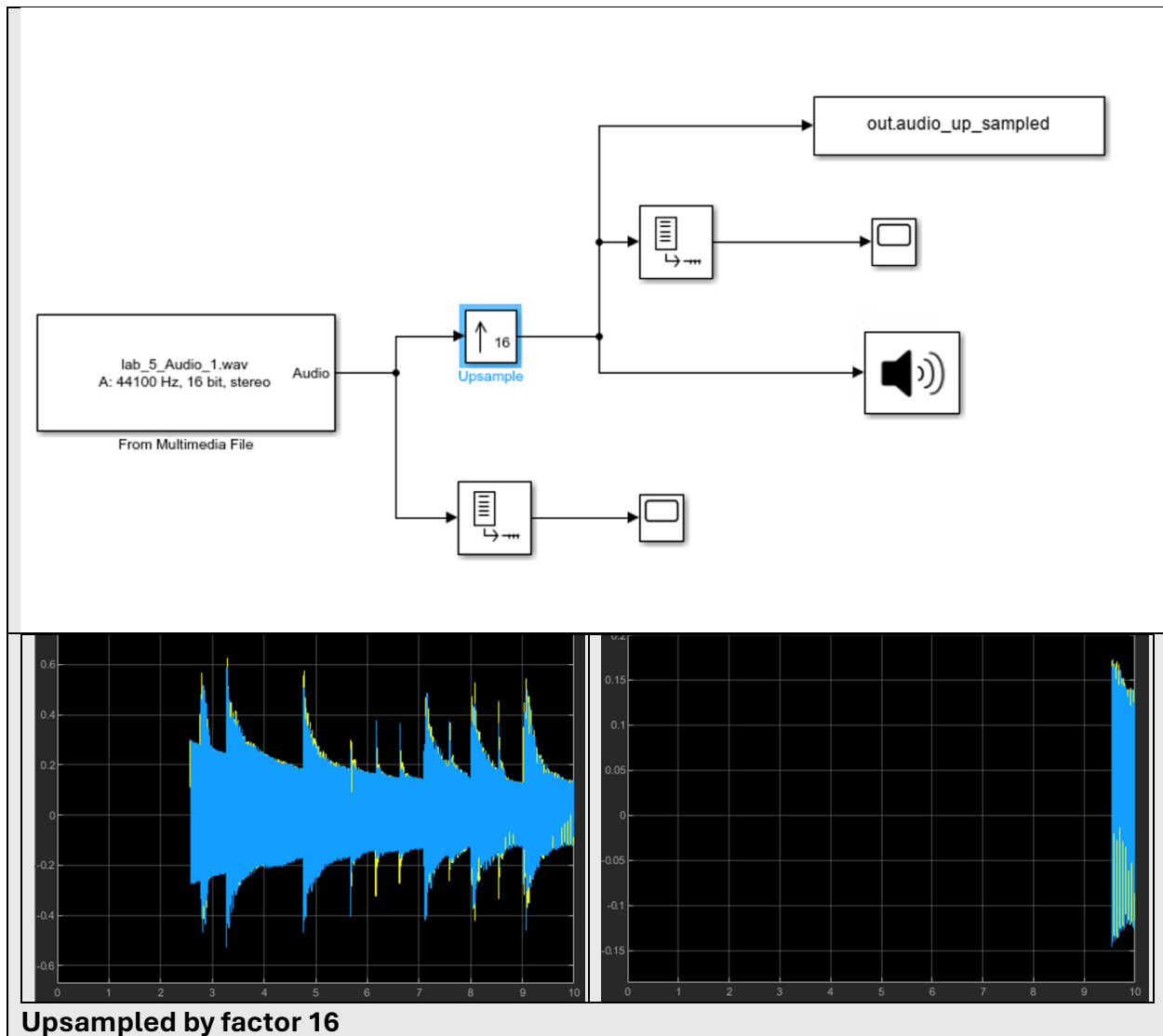




The Down sampling block reduced the sampling rate, which made the song less and less recognizable. With a factor of 8, the sound quality degraded slightly and high frequencies were lost. With a factor of 16, the degradation was clearly noticeable to the point where it was unrecognisable and the signal's frequency content was further reduced.

Answer (3B):





The up sampling block raised the sampling rate, which made the song less and less recognizable. With a factor of 8, the sound quality increased slightly and high frequencies were a little more intense. With a factor of 16, the degradation was clearly noticeable to the point where it was unrecognisable and the signal's frequency content was further increased, but the high pitch made the song unrecognisable.

Conclusions –

In this lab, we explored the fundamental concepts of digital and analog filter design using MATLAB and Simulink, focusing on low-pass, high-pass, band-pass, and notch filters. We created, analyzed, and processed audio signals by implementing both upsampling and downsampling techniques, visualizing their impact in time and frequency domains. Additionally, the use of tools such as FDATool enabled efficient filter design and signal analysis. Overall, the lab provided valuable insights into the interplay between signal processing techniques and real-world implementation in a dynamic simulation environment.